# ЛАБОРАТОРНАЯ РАБОТА № 3

# ИИ генерирует README, changelog и API-документацию в Markdown

**Программное обеспечение:** GigaChat / YandexGPT, VS Code, Markdown.

**Форма отчётности:** ЛР3_Отчёт_Фамилия.pdf + файлы README.md, CHANGELOG.md, API.md, example.py.

**Цель:** научиться создавать проектную документацию в формате Markdown с применением ИИ-инструментов и проверять её на соответствие принятым стандартам оформления.

### Рабочее место: где и как выполнять работу

Работа выполняется в VS Code – бесплатном редакторе кода, который дальше используется на протяжении всего курса. Ничего запускать и компилировать не нужно: README.md, CHANGELOG.md и API.md – обычные текстовые файлы, а VS Code умеет сразу показывать, как они будут выглядеть при открытии на GitHub.

1. Скачайте VS Code с официального сайта code.visualstudio.com и установите его, если он ещё не установлен (в лабораторной работе № 5 он понадобится с расширениями, но базовая установка не занимает много времени и её можно сделать уже сейчас).
2. Откройте VS Code и через меню Файл → Открыть папку выберите папку taskflow, созданную в предыдущей лабораторной работе (если папки ещё нет – создайте её на рабочем столе).
3. Создайте три новых файла прямо в этой папке: правой кнопкой мыши по названию папки в панели слева → «Новый файл» → введите имя README.md. Так же создайте CHANGELOG.md, API.md и example.py.
4. Текст, полученный от ИИ, копируйте прямо в соответствующий файл и сохраняйте сочетанием клавиш Ctrl+S.

> **Как посмотреть готовый результат:** откройте файл с расширением .md и нажмите Ctrl+Shift+V (или значок с лупой и книгой в правом верхнем углу вкладки) – откроется предпросмотр, в котором заголовки, списки и таблицы отображаются так же, как они будут выглядеть на GitHub.

Файл example.py в этой работе тоже не запускается – важно только, чтобы код и добавленные к нему комментарии были читаемыми. Если хочется убедиться, что функция действительно работает, можно (по желанию) установить Python с сайта python.org и выполнить файл командой python example.py в терминале VS Code (Ctrl+`), но для выполнения самой лабораторной работы это не обязательно.

### Теоретическая справка

### Markdown как язык разметки

Markdown придумали, чтобы можно было форматировать текст (заголовки, списки, таблицы, ссылки), не открывая при этом никакого специального редактора вроде Word – достаточно обычного текстового файла с расширением .md. Специальные символы вокруг слов задают оформление, а программа, которая умеет читать Markdown (GitHub, Gitflic, VS Code, Jupyter Notebook), превращает эти символы в привычный форматированный текст.

| Что нужно | Как написать | Как отобразится |
|---|---|---|
| Заголовок | ## Установка | крупный полужирный текст-заголовок |

| Что нужно | Как написать | Как отобразится |
|---|---|---|
| Полужирный текст | **важно** | важно |
| Список | - пункт один | маркированный список |
| Ссылка | [текст](https://example.com) | кликабельная ссылка |
| Блок кода | ```python print(1) ``` | код моноширинным шрифтом с подсветкой |
| Таблица | \| A \| B \| \|---\|---\| \| 1 \| 2 \| | таблица с колонками A и B |

**Что должно быть в README.md**

README.md – первый файл, который открывает любой человек, попавший в репозиторий, поэтому от него зависит, поймут ли вообще, что это за проект и как его запустить. В хорошем README обычно есть: короткое описание проекта (одно-два предложения), список технологий (стек), пошаговая инструкция установки и запуска, структура основных папок, ссылка на подробную документацию API и указание лицензии. Значки (badges) от сервиса Shields.io – не обязательный, но распространённый способ сразу показать версию Python, лицензию и статус сборки проекта.

**CHANGELOG.md и формат Keep a Changelog**

CHANGELOG.md – это история версий проекта: что изменилось от релиза к релизу. Формат Keep a Changelog задаёт единый способ записи: каждая версия оформляется как заголовок вида ## [1.0.0] - 2026-03-01, а изменения внутри неё группируются по разделам Added (что добавили), Changed (что изменили), Fixed (что исправили), Removed (что убрали). Такой файл гораздо удобнее читать, чем историю коммитов Git: он написан для людей, а не для машин.

**Документирование API**

Для каждого эндпоинта в API.md обычно указывают одно и то же: HTTP-метод и адрес, что делает запрос, какие данные нужно отправить в теле запроса и что сервер вернёт в ответ. Пример оформления одного эндпоинта:

```
### POST /auth/register
Регистрация нового пользователя.

Тело запроса:
{ "username": "anna", "email": "anna@mail.ru", "password": "..." }

Ответ (201 Created):
{ "id": 1, "username": "anna" }
```

Такой Markdown-файл читается быстро и не требует запуска проекта, в отличие от Swagger/OpenAPI, где документация строится автоматически из кода и требует запущенного сервера – зато сама отражает актуальное состояние API и позволяет отправлять запросы прямо из браузера. Для учебного и небольшого проекта Markdown-версии обычно достаточно.

**Docstring в Python**

Docstring – это строка документации сразу после объявления функции, которая объясняет, что функция делает, какие параметры принимает и что возвращает. В отличие от обычного комментария (#), docstring можно посмотреть в самой программе через функцию help() или во

всплывающей подсказке редактора. Самые распространённые стили оформления – Google style и reStructuredText; для учебного проекта достаточно придерживаться одного стиля во всех файлах.

**Ход работы**

**Шаг 1. Подготовьте рабочее место**

1. Откройте папку taskflow в VS Code (см. раздел «Рабочее место» выше).
2. Создайте файл отчёта и укажите фамилию, имя, группу, дату и тему работы.

**Шаг 2. Сгенерируйте README.md**

В GigaChat или YandexGPT отправьте промпт:

> Создай README.md для проекта TaskFlow – REST API менеджера задач на Python/Flask, SQLite. Включи: описание, стек (Python, Flask, SQLAlchemy, SQLite), инструкцию по установке (pip install, python app.py), раздел API (ссылка на API.md), лицензию MIT. Используй значки Shields.io для Python и License.

Сохраните результат как README.md. Откройте предпросмотр (Ctrl+Shift+V) и проверьте результат по чек-листу:

| Проверка README.md | Да / частично / нет | Комментарий |
|---|---|---|
| Есть краткое и понятное описание проекта | | |
| Указан стек технологий | | |
| Инструкция по установке и запуску написана по шагам | | |
| Есть раздел или ссылка на API-документацию | | |
| Указана лицензия и добавлены значки Shields.io | | |

Исправьте найденные недочёты вручную прямо в файле.

**Шаг 3. Сгенерируйте CHANGELOG.md**

Отправьте промпт:

> Создай CHANGELOG.md для проекта TaskFlow в формате Keep a Changelog. Добавь три версии: 0.1.0 (начальная структура проекта), 0.2.0 (добавлен CRUD для задач), 1.0.0 (авторизация, Docker, деплой).

Сохраните результат как CHANGELOG.md. Проверьте, что каждая версия оформлена заголовком вида ## [версия] - дата и содержит хотя бы один из разделов Added / Changed / Fixed.

**Шаг 4. Создайте API.md**

Создайте файл API.md вручную или с помощью ИИ. Минимальный состав документа:

1. Авторизация: POST /auth/register, POST /auth/login;
2. Задачи: GET/POST /tasks, GET/PUT/DELETE /tasks/{id};
3. для каждого эндпоинта – метод, URL, тело запроса (JSON) и пример ответа (см. образец оформления в теоретической справке).

### Шаг 5. Задокументируйте фрагмент кода

Напишите простую Python-функцию (например, get_tasks()) и попросите ИИ добавить docstring и построчные комментарии. Вставьте результат в файл example.py.

### Шаг 6. Проверьте и оформите отчёт

Откройте предпросмотр всех трёх .md-файлов в VS Code и сделайте скриншоты. Убедитесь, что таблицы и блоки кода отображаются корректно, а не как обычный неотформатированный текст.

### Оформление отчёта

Сохраните отчёт под именем ЛР3_Отчёт_Фамилия.pdf. Рекомендуемая структура:

1. Титульная часть: Ф. И. О., группа, дата, тема.
2. Промпт и содержимое README.md, заполненный чек-лист.
3. Промпт и содержимое CHANGELOG.md.
4. Содержимое API.md с описанием минимум четырёх эндпоинтов.
5. Функция example.py с docstring и комментариями, добавленными ИИ.
6. Скриншоты предпросмотра всех трёх .md-файлов в VS Code.
7. Ответы на контрольные вопросы.
8. Итоговый вывод объёмом 5–7 предложений.

**В выводе ответьте:** что из сгенерированного ИИ пришлось исправить вручную; почему нельзя публиковать README без проверки инструкции по установке; чем Markdown-документация уступает Swagger/OpenAPI и в чём её преимущество.

### Контрольные вопросы

1. Чем Markdown отличается от HTML? В каких случаях предпочтительнее каждый?
2. Что обязательно должно быть в README.md профессионального open-source проекта?
3. Что такое Keep a Changelog? Какова структура записи о версии?
4. Как вставить блок кода с подсветкой синтаксиса в Markdown?
5. Что такое docstring в Python? Какой стандарт оформления применяется?
6. В чём разница между Swagger/OpenAPI и ручной документацией в Markdown?
7. Как ИИ может ошибиться при генерации документации? Что нужно проверять?

### Работа считается выполненной, если

1. получены и сохранены README.md, CHANGELOG.md и API.md;
2. README.md проверен по чек-листу, найденные недочёты исправлены;

3. CHANGELOG.md оформлен по формату Keep a Changelog с тремя версиями;
4. API.md описывает минимум четыре эндпоинта с телом запроса и примером ответа;
5. фрагмент кода в example.py снабжён docstring и комментариями;
6. даны ответы на контрольные вопросы и сформулирован самостоятельный вывод;
7. отчёт сохранён в PDF, все .md-файлы приложены отдельно.

*Критерии выставления отметки – согласно сводной таблице 3 (раздел 2.1).*

**Полезные ссылки**

1. VS Code: https://code.visualstudio.com
2. GigaChat: https://giga.chat
3. Сервис Яндекса с ИИ: https://ya.ru/ai
4. Keep a Changelog: https://keepachangelog.com
5. Shields.io: https://shields.io
6. Markdown Guide: https://www.markdownguide.org
7. Swagger/OpenAPI: https://swagger.io